

Event & Message Design

1. Event Envelope (all events)

```
{
  "eventId": "uuid",           // consumer dedupe key
  "eventType": "PaymentSucceeded",
  "aggregateId": "uuid",
  "occurredAt": "2026-10-05T11:03:12Z",
  "correlationId": "trace-id",
  "version": 1,
  "payload": { }
```

Delivery: **at-least-once** via transactional outbox → Kafka. Ordering guaranteed per partition key. All consumers are idempotent (`processed_event`).

2. Commands vs Events

Type	Name	Sender → Receiver	Style
Command	ReserveInventory	Checkout/Gateway → Inventory	Sync API
Command	InitiatePayment	Checkout → Payment	Sync API
Command	ConfirmReservation / ReleaseReservation	Saga → Inventory	Async/sync
Command	CreateShipment	Order event → Shipment	Async
Event	InventoryReserved	Inventory	Async
Event	ReservationExpired / ReservationReleased	Inventory	Async
Event	InventorySold	Inventory	Async
Event	StockExhausted	Inventory	Async
Event	PaymentInitiated / PaymentSucceeded / PaymentFailed / PaymentTimedOut	Payment	Async
Event	RefundIssued	Payment	Async
Event	OrderCreated / OrderConfirmed / OrderCancelled	Order	Async
Event	OrderCreationFailed	Order/Recovery	Async
Event	ShipmentCreated / ShipmentStatusChanged / Delivered	Shipment	Async
Event	NotificationSent / NotificationFailed	Notification	Async

3. Event Ownership & Consumers

Event	Owner (only publisher)	Partition key	Consumers & responsibility
InventoryReserved	Inventory	product_id	Notification (optional "reserved"), Analytics, Waitlist
ReservationExpired / Released	Inventory	product_id	Waitlist service (re-offer unit), Notification, Cache updater (sold-out flag)
StockExhausted	Inventory	product_id	Cache/CDN (show Sold Out), Admission (stop admitting)
PaymentSucceeded	Payment	reservation_id	Inventory (confirm), Order (create order), Checkout saga
PaymentFailed / PaymentTimedOut(final)	Payment	reservation_id	Inventory (release), Order (cancel), Notification
OrderConfirmed	Order	order_id	Inventory (mark SOLD), Shipment (create), Notification (email/SMS)
OrderCancelled	Order	order_id	Payment (refund), Inventory (release/restock), Notification
OrderCreationFailed	Order/Recovery	reservation	Payment (refund), Inventory (release)

		_id	
ShipmentStatusChanged	Shipment	order_id	Order (state advance), Notification

4. Topics

inventory.events, payment.events, order.events, shipment.events, notification.events, plus *.dlq per consumer group. Retention 7 days (replay); compacted topic for latest state snapshots where useful.

5. Failure Handling for Messages

1. Consumer error → retry 3 times with exponential backoff (local retry topic).
2. Still failing → publish to <topic>.dlq with error metadata; alert if DLQ depth > 0.
3. DLQ replay tool re-injects after fix; idempotency makes replay safe.
4. Poison messages never block the partition (moved to DLQ after retries).
5. Consumer lag is monitored; autoscale by lag.